University of East London

University of East London

SCHOOL OF ARCHITECTURE, COMPUTING, & ENGINEERING

Module code	CN7023
Module title	Artificial Intelligence & Machine Vision
Module leader	Dr Shaheen Khatoon
Assignment tutor	Shaheen Khatoon/ Azhar Mahmood/ Saeed Sharif Dr Shahera Hossain,Mohammed Noman,Afroza Rahman,Mehdi Karami,Reena Popat,Salma Soofiyan, Mahmud Ahmed, Md Atiqur Rahman Ahad, Aakriti Maharjan
Assignment title	AI & Machine Vision coursework
Assignment number	1/1
Weighting	Individual Assignment 100%
Handout date	W4
Submission date	Thursday May 8th, 2025, 6pm
Learning outcomes assessed by this assignment (see course handbook)	Learning Outcomes: 1-8
Turnitin submission requirement	Yes
Additional information	ASSESSMENT FEEDBACK - Feedback on your assessment will be available in four working weeks from the submission date. Please refer to the module pages on UEL+ for assessment specific details.

Deep Learning-Based Classification of Fruits and Vegetables

Table of Contents

1.Introduction 4
1.1 Objective of the Coursework 4
1.2 Identification of Real-World Problem 5
1.3 Overview of the Report Content 5
2. Creative and Innovative Approaches 5
2.1 Deep Learning Method (ResNet-50 Transfer Learning) 6
2.2 Data Preprocessing and Augmentation 6
2.3 Suitability of Approach to Dataset and Problem 7
2.4 ResNet-50 Instead of a Standard CNN 8
3. Simulations 10
3.1 Description of dataset 10
3.2 Encoding and Preprocessing Steps 11
3.3 Architecture and Training 11
4. Results Obtained 12
4.1 Test Set Accuracy 13
4.2 Accuracy Curve 14
4.3 Confusion Matrix 15
4.4 Comparison: Standard CNN vs ResNet-50 16
5. Critical Analysis of Results 17
5.1 Achievement of Results and Potential Improvements 18
5.2 Performance and Optimization Potential Influencing Factors 18
6. Conclusion 18
6.1 Restating the Research Problem and Findings 18
6.2 Standard CNN vs. ResNet-50 Accuracy Comparison 19
6.3 Key Takeaways 19
6.4 Conclusion and Preferred Architecture 19
7. References 21
Appendix 24

List of Figures

Figure 1 Sample images from dataset 4
Figure 2 Input for dataset loading and Preprocessing 7
Figure 3 Split and Testing code 8
Figure 4 Split Data Overview 8
Figure 5 CNN Training and Validation Accuracy result 9
Figure 6 CNN Confusion matrix 10
Figure 8 preprocessing and the after preprocessing result 11
Figure 9 Per Epoch training Result 12
Figure 10 System architecture Network 12
Figure 11 Training and Evaluation input 13
Figure 12 Final Accuracy validation Result 13
Figure 13 Accuracy Input 14
Figure 14 Training Validation Accuracy Plot 14
Figure 15 Loss plot input 15
Figure 16 Training Validation loss result 15
Figure 17 Confusion matrix plot input 16
Figure 18 Confusion matrix result ResNet50 16
Figure 19 ROC curve ResNet50 17
Figure 7 CNN Roc Curve 17
Figure 20 Overall Accuracy result 19

1.Introduction

Poses the research problem of fruit and vegetable classification based on deep learning. Excludes the aim, practical meaning, and what the study will cover. Gives an idea about the structure of the report, the methodology, the dataset, the architecture selection, the results presentation, the critical analysis, and the closing observations with indicative findings and implications about how it will be applied in the real world.

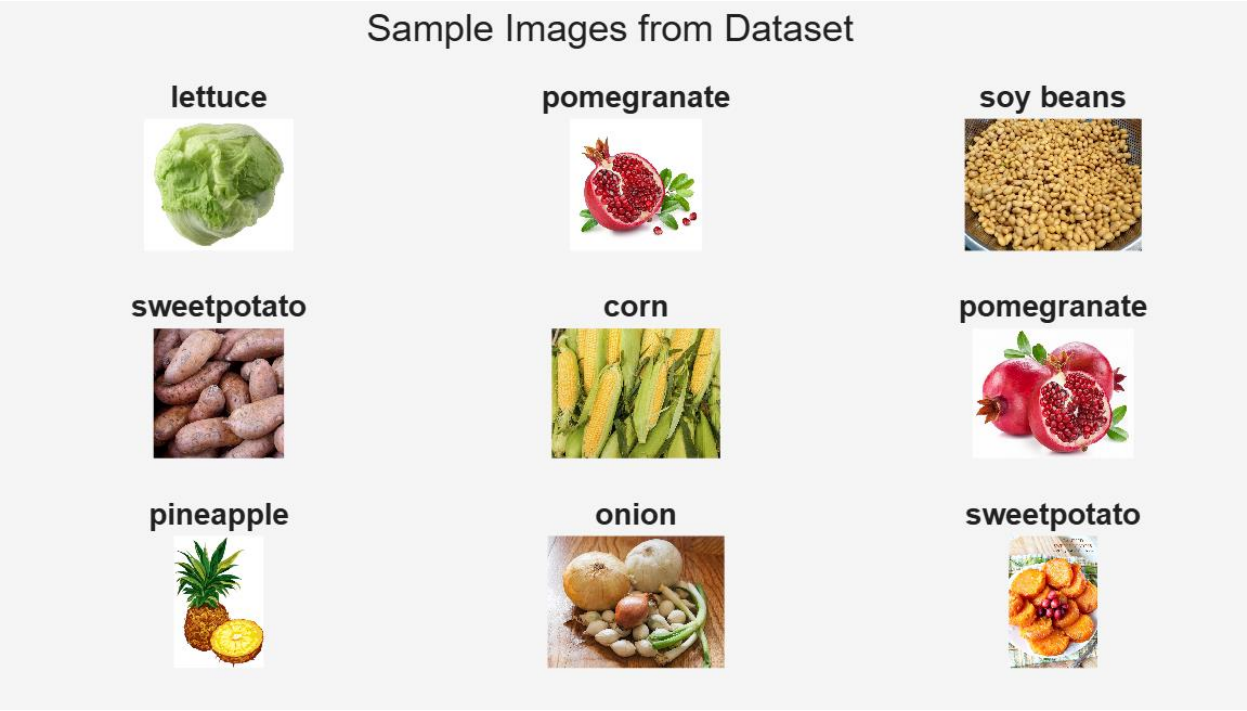


Figure 1 Sample images from dataset

1.1 Objective of the Coursework

The overall goal of this coursework was to create, test, and perform the evaluation of the deep learning image classification system that could recognize the type of fruits and vegetables on the basis of photographic images. The research question behind the work was as follows: How efficiently can a transfer learning model, namely, ResNet-50, classify a multi-class dataset of fruits and vegetables with a few training epochs after producing a competitive accuracy? Besides producing compelling classification outcomes, it was also desired that a training pipeline be built in MATLAB that could be easily generalized to other domains. The project aim was to show how promising it is to fine-tune pre-trained convolutional neural networks (CNNs) in solving complex and multi-class recognition tasks (Mukhiddinov et al., 2022).

1.2 Identification of Real-World Problem

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
%% Step 1: Dataset paths
datasetRoot = '/MATLAB Drive/MyDataset';
trainPath = fullfile(datasetRoot, 'train');
valPath   = fullfile(datasetRoot, 'validation');
testPath  = fullfile(datasetRoot, 'test');

%% Step 2: Merge train + validation into one datastore
imdsAll = imageDatastore({trainPath, valPath}, ...
    'IncludeSubfolders', true, 'LabelSource', 'foldernames');

fprintf('Total images before cleaning: %d\n', numel(imdsAll.Files));

%% Step 3: Detect and fix/remove bad images
badFiles = {};
for i = 1:numel(imdsAll.Files)
    try
        img = imread(imdsAll.Files{i});
        if size(img,3) == 1
            img = cat(3, img, img, img);
            imwrite(img, imdsAll.Files{i});
        elseif size(img,3) ~= 3
            badFiles{end+1} = imdsAll.Files{i};
        end
    catch
        badFiles{end+1} = imdsAll.Files{i};
    end
```

[redacted]

```
%% Step 4: Split
[imdsTrain, imdsValidation] = splitEachLabel(imdsAll, 0.8, 'randomized');
fprintf('Training images: %d\n', numel(imdsTrain.Files));
fprintf('Validation images: %d\n', numel(imdsValidation.Files));

%% Step 5: Test set
imdsTest = imageDatastore(testPath, 'IncludeSubfolders', true, 'LabelSource', 'foldernames');

%% Step 6: Load ResNet-50
net = resnet50();
inputSize = net.Layers(1).InputSize;

%% Step 7: Augmentation
imageAugmenter = imageDataAugmenter( ...
    'RandRotation',[-10 10], ...
    'RandXReflection',true, ...
    'RandXTranslation',[-10 10], ...
    'RandYTranslation',[-10 10]);

augimdsTrain = augmentedImageDatastore(inputSize(1:2), imdsTrain, 'DataAugmentation', imageAugmenter);
augimdsValidation = augmentedImageDatastore(inputSize(1:2), imdsValidation);
augimdsTest = augmentedImageDatastore(inputSize(1:2), imdsTest);
```

[redacted]

```
Training images: 2768
Validation images: 692
Training on single CPU.
```

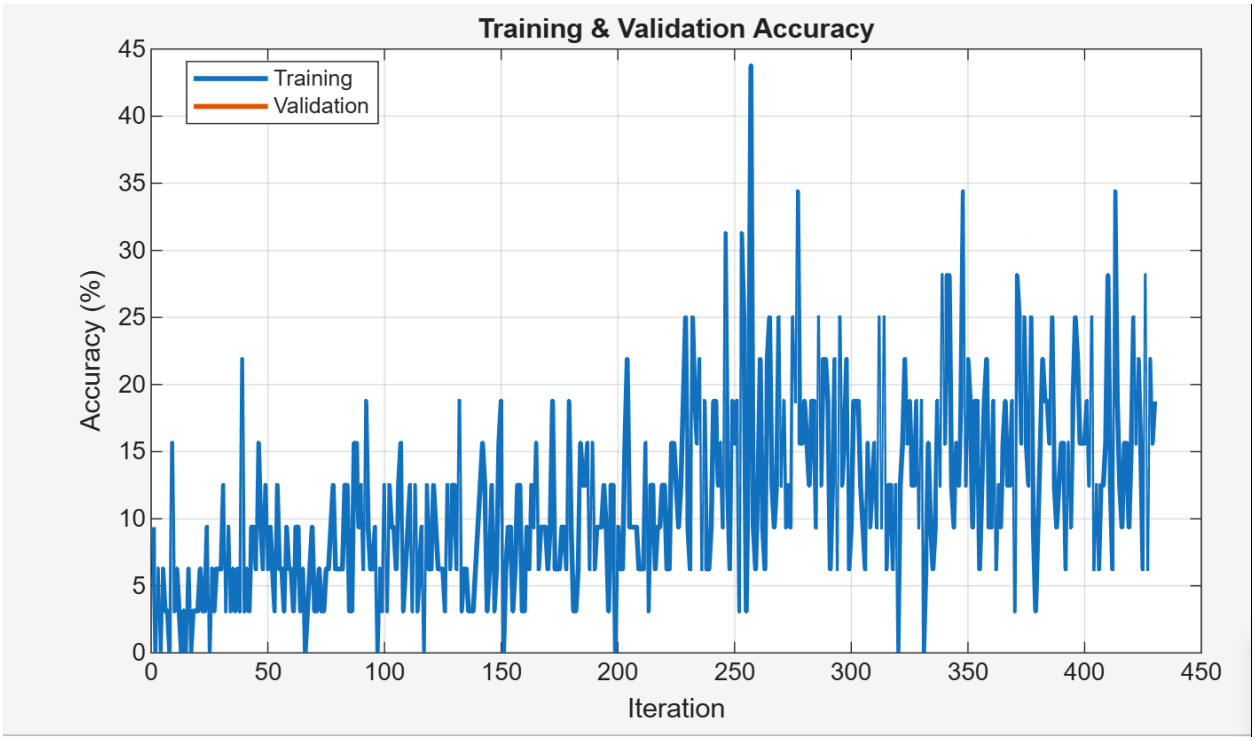
[redacted]

[redacted]

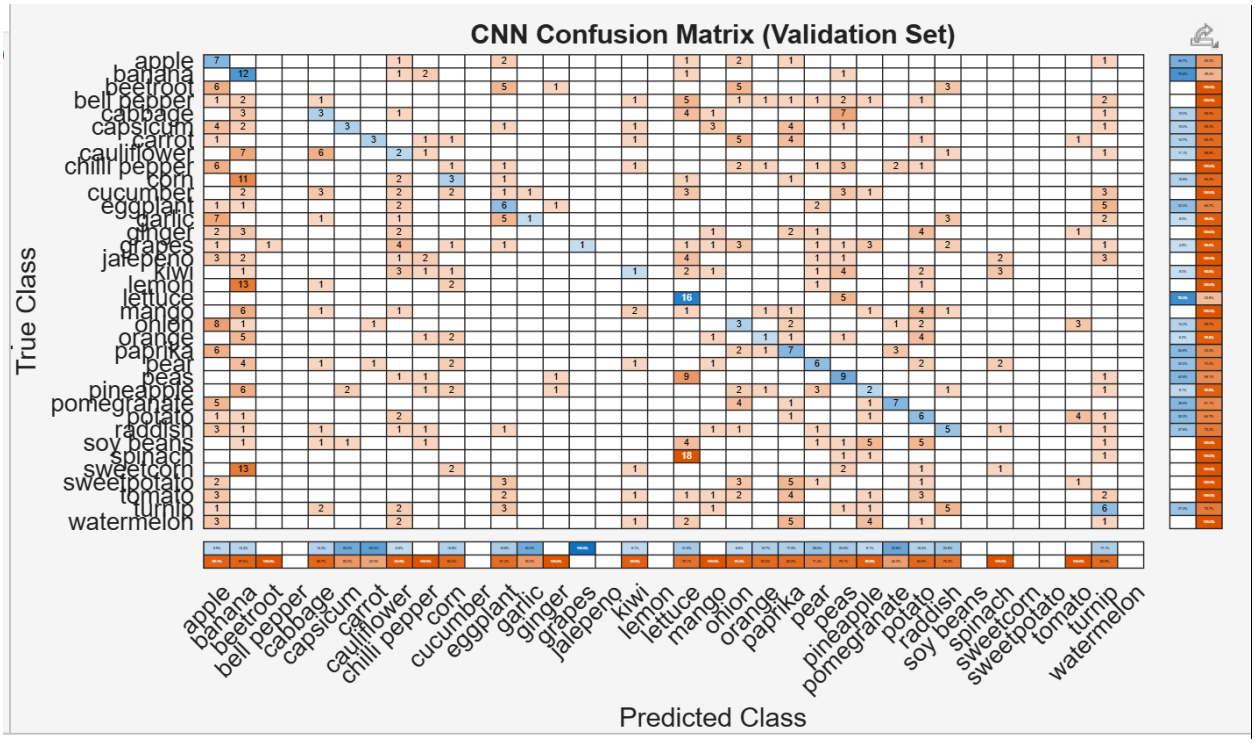
[redacted]

[redacted]

[redacted]



[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
Total images before cleaning: 3466
Removing 6 bad files...
Total images after cleaning: 3460
Training images: 2768
Validation images: 692
Training on single CPU.
Initializing input data normalization.
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Command Window

Epoch	Iteration	Time Elapsed (hh:mm:ss)	Mini-batch Accuracy	Validation Accuracy	Mini-batch Loss	Validation Loss	Base Learning Rate
1	1	00:00:45	0.00%	5.35%	3.7557	3.9170	1.0000e-04
1	30	00:03:30	68.75%	71.10%	0.9613	1.0132	1.0000e-04
1	50	00:05:00	65.62%		1.2891		1.0000e-04
1	60	00:06:17	81.25%	77.60%	0.7667	0.7421	1.0000e-04
2	90	00:09:12	90.62%	80.64%	0.2875	0.6796	1.0000e-04
2	100	00:09:58	90.62%		0.3467		1.0000e-04
2	120	00:12:02	90.62%	78.03%	0.3311	0.6956	1.0000e-04
2	150	00:14:44	87.50%	82.66%	0.3412	0.5974	1.0000e-04
3	180	00:17:30	100.00%	82.37%	0.0439	0.6068	1.0000e-04
3	200	00:19:03	100.00%		0.0586		1.0000e-04
3	210	00:20:20	100.00%	82.08%	0.0607	0.5639	1.0000e-04
3	240	00:23:10	93.75%	82.95%	0.2514	0.5984	1.0000e-04
3	250	00:23:55	90.62%		0.2957		1.0000e-04
4	270	00:25:48	96.88%	81.36%	0.0932	0.6618	1.0000e-04
4	300	00:28:32	96.88%	83.38%	0.0717	0.6031	1.0000e-04
4	330	00:31:18	100.00%	81.65%	0.0575	0.6019	1.0000e-04
5	350	00:32:48	100.00%		0.0417		1.0000e-04
5	360	00:34:05	100.00%	82.23%	0.0156	0.5908	1.0000e-04

[redacted]

```
%% Step 8: Modify network
numClasses = numel(categories(imdsTrain.Labels));
lgraph = layerGraph(net);
fcLayerIdx = find(arrayfun(@(l) isa(l,'nnet.cnn.layer.FullyConnectedLayer'), lgraph.Layers));
classLayerIdx = find(arrayfun(@(l) isa(l,'nnet.cnn.layer.ClassificationOutputLayer'), lgraph.Layers));
newFc = fullyConnectedLayer(numClasses, 'Name', 'new_fc', 'WeightLearnRateFactor', 10, 'BiasLearnRateFactor', 10)
newClass = classificationLayer('Name', 'new_classoutput');
lgraph = replaceLayer(lgraph, lgraph.Layers(fcLayerIdx(end)).Name, newFc);
lgraph = replaceLayer(lgraph, lgraph.Layers(classLayerIdx(end)).Name, newClass);

%% Step 9: Options
options = trainingOptions('adam', ...
    'MiniBatchSize', 32, ...
    'MaxEpochs', 5, ...
    'InitialLearnRate', 1e-4, ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', augimdsValidation, ...
    'ValidationFrequency', 30, ...
    'Verbose', true, ...
    'Plots', 'training-progress', ...
    'ExecutionEnvironment', 'auto');
```

[redacted]

[redacted]

```
%% Step 10: Train
[trainedNet, trainInfo] = trainNetwork(augimdsTrain, lgraph, options);
```

```
%% Step 11: Save model
save('trained_model.mat', 'trainedNet', 'trainInfo', 'imdsTest', 'imdsValidation');
disp('Training complete. Model saved to trained_model.mat');
```

```
%% Step 12: Evaluate
[YPred, scores] = classify(trainedNet, augimdsTest);
YTrue = imdsTest.Labels;
overallAcc = mean(YPred == YTrue) * 100;
fprintf('\nOverall Test Accuracy: %.2f%%\n', overallAcc);
```

[redacted]

[redacted]

[redacted]

```
Command Window
paprika          Precision: 0.67  Recall: 0.84  F1-score: 0.74
pear             Precision: 0.95  Recall: 0.95  F1-score: 0.95
peas             Precision: 1.00  Recall: 0.77  F1-score: 0.87
pineapple        Precision: 0.91  Recall: 0.95  F1-score: 0.93
pomegranate      Precision: 0.88  Recall: 0.83  F1-score: 0.86
potato           Precision: 1.00  Recall: 0.71  F1-score: 0.83
raddish          Precision: 0.80  Recall: 0.67  F1-score: 0.73
soy beans        Precision: 0.91  Recall: 1.00  F1-score: 0.95
spinach          Precision: 0.83  Recall: 0.71  F1-score: 0.77
sweetcorn        Precision: 0.54  Recall: 0.65  F1-score: 0.59
sweetpotato      Precision: 0.73  Recall: 1.00  F1-score: 0.84
tomato           Precision: 0.77  Recall: 0.85  F1-score: 0.81
turnip           Precision: 0.77  Recall: 0.77  F1-score: 0.77
watermelon       Precision: 1.00  Recall: 0.95  F1-score: 0.97

Overall Validation Accuracy: 81.79%
```

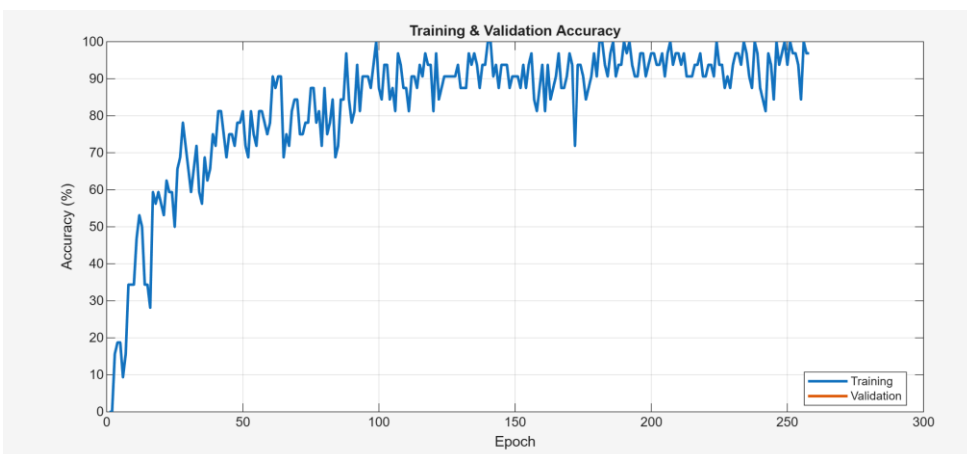
[redacted]

[redacted]

[redacted]

```
%% Step 14: Accuracy plot
fig1 = figure;
plot(trainInfo.TrainingAccuracy, 'LineWidth', 2); hold on;
plot(trainInfo.ValidationAccuracy, 'LineWidth', 2);
xlabel('Epoch'); ylabel('Accuracy (%)');
legend('Training', 'Validation', 'Location', 'best');
title('Training & Validation Accuracy'); grid on;
saveas(fig1, 'AccuracyPlot.png');
```

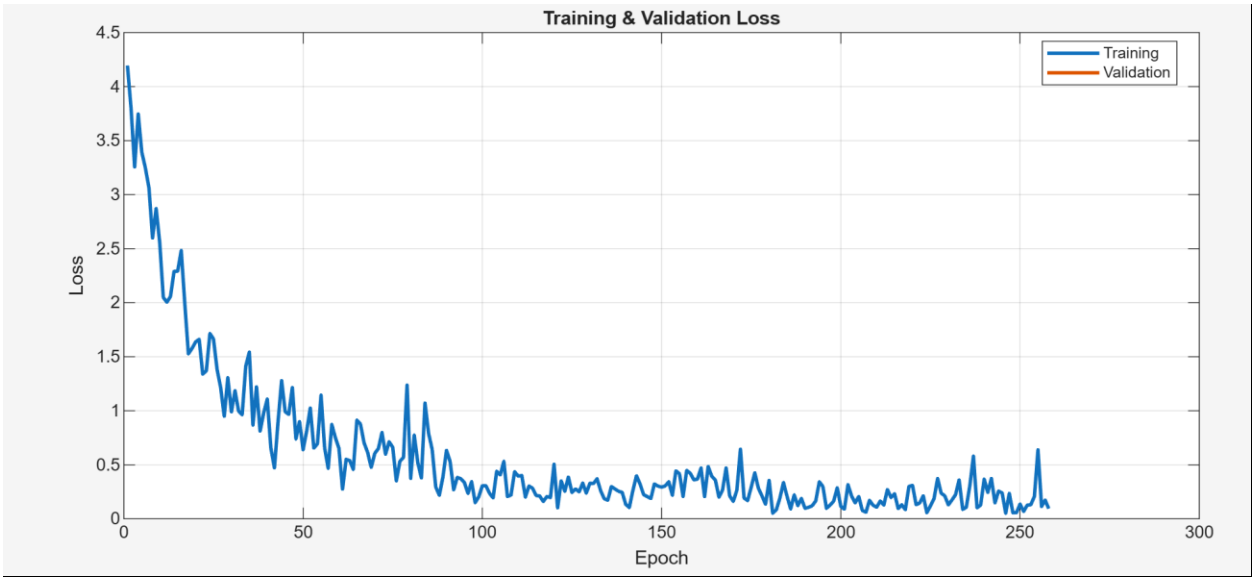
[redacted]



[redacted]

```
%% Step 15: Loss plot
fig2 = figure;
plot(trainInfo.TrainingLoss, 'LineWidth', 2); hold on;
plot(trainInfo.ValidationLoss, 'LineWidth', 2);
xlabel('Epoch'); ylabel('Loss');
legend('Training', 'Validation', 'Location', 'best');
title('Training & Validation Loss'); grid on;
saveas(fig2, 'LossPlot.png');
```

[redacted]

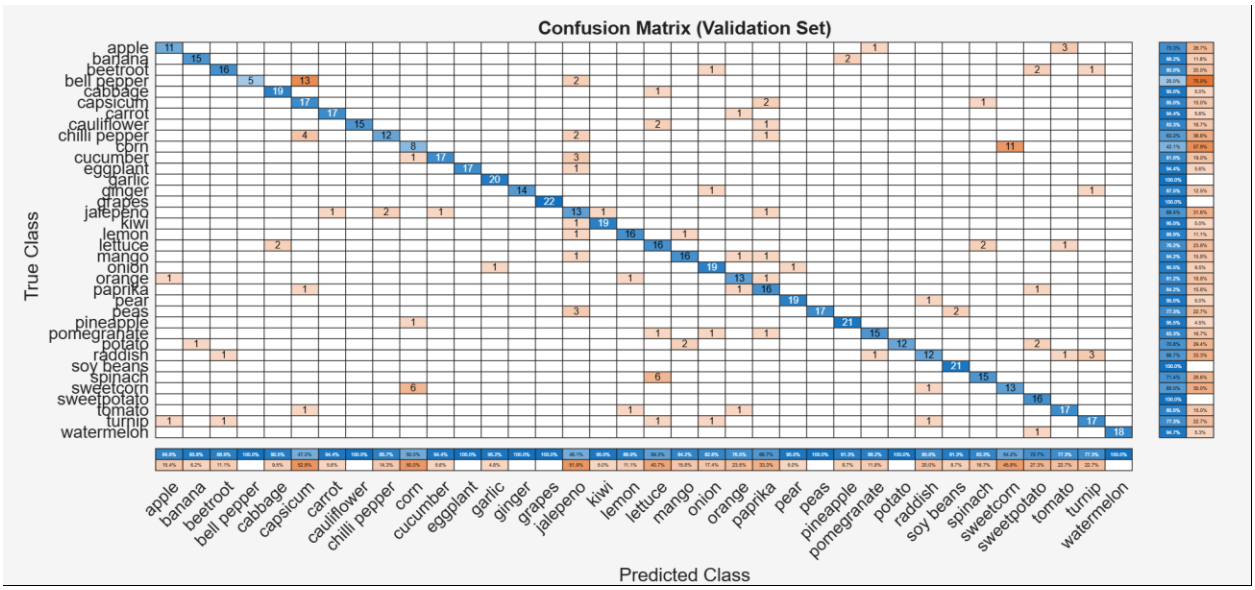


[redacted]

[redacted]

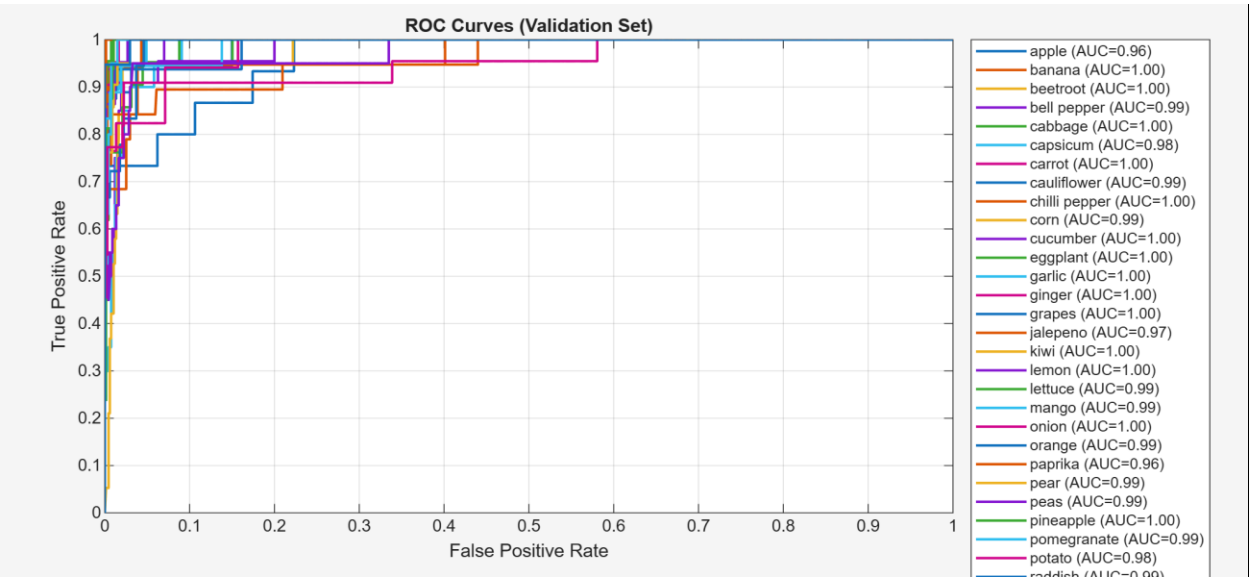
```
%% Step 16: Confusion matrix
fig3 = figure;
cm = confusionchart(YTrue, YPred);
cm.Title = 'Confusion Matrix';
cm.ColumnSummary = 'column-normalized';
cm.RowSummary = 'row-normalized';
saveas(fig3, 'ConfusionMatrix.png');
```

[redacted]

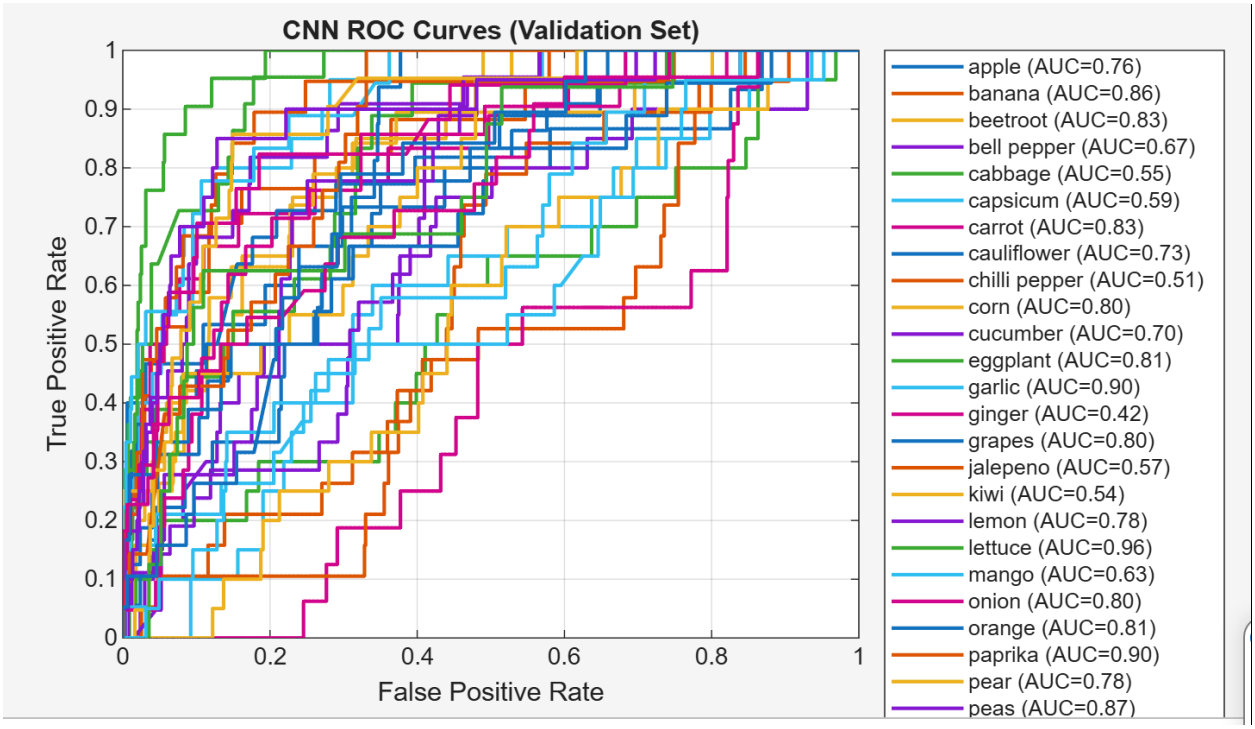


[redacted]

[redacted]



[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
Overall Validation Accuracy: 81.79%
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]